# Lab 1: Know Your Machine & GitHub Foundations

PPOL 5206 Massive Data Fundamentals

# Today's Objectives

By the end of this lab, you will:

- Discover and document your computer's configuration
- Understand how PATH determines which programs run
- Install and configure Git for version control
- Create a GitHub repository and publish it as a live website
- Practice the commit-push workflow

# Why This Matters

"Command not found" is the #1 error you'll encounter

- Before you can:
- Connect to cloud databases
- Run Spark jobs
- Deploy machine learning models

You need to understand your own machine

# Today's Roadmap

| Part | Topic |
| --- | --- |
| 1 | Know Your Machine |
| 2 | Install & Configure Git |
| 3 | GitHub Authentication |
| 4 | Create Your Repository |
| 5 | Publish with GitHub Pages |

# Part 1:
# Know Your Computer

# PowerShell

For Windows and Mac:

https://github.com/PowerShell/PowerShell

Source: Citation goes here

# Opening Your Terminal

**Windows:**
Press Windows key → Type "PowerShell“
**Right-click** and select **'Run as Administrator'**

**Mac:**
Press Cmd + Space → Type "Terminal" → Enter
Or: Applications → Utilities → Terminal

- Keep this open for the entire lab!
- Sometimes, to apply settings, closing and reopening the terminal can help
- (similar to restarting your computer).

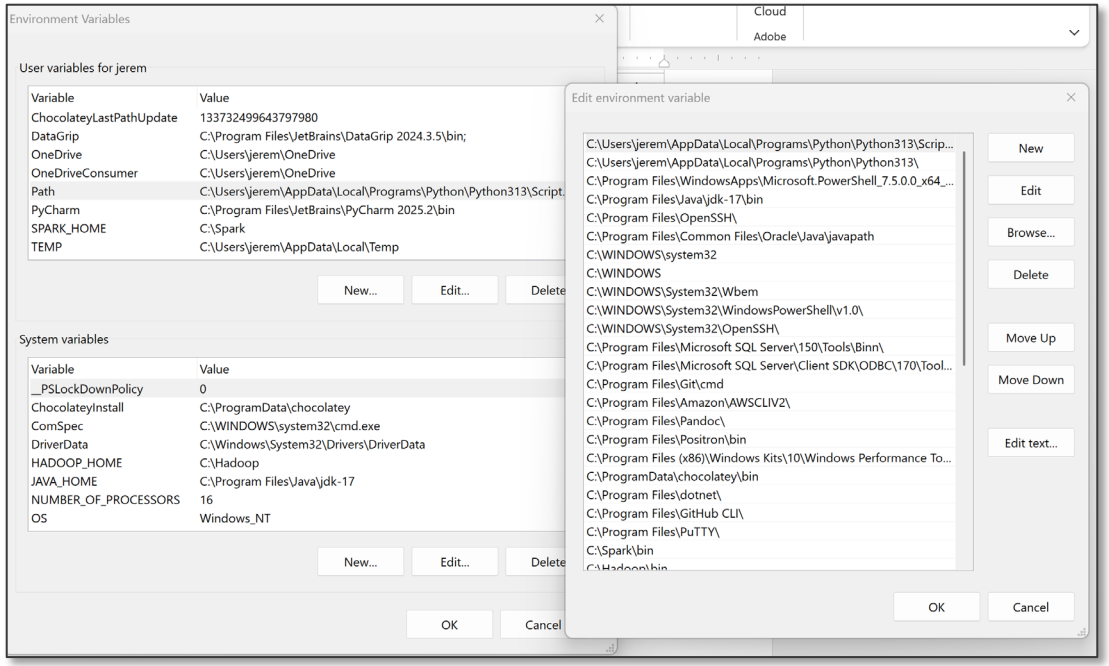
# Discover Your System

| What | Windows | Mac |
|---|---|---|
| Operating System | (Get-CimInstance Win32_OperatingSystem).Caption | sw_vers |
| Architecture | $env:PROCESSOR_ARCHITECTURE | uname -m |
| Username | $env:USERNAME | whoami |
| Home Directory | $env:USERPROFILE | echo $HOME |

Record each value in your checklist!

# Understanding PATH

When you type a command like python…
Your system searches a list of directories to find it



# View Your PATH

**Windows (PowerShell):**

```
$env:PATH -split ‘;’
```

**Mac/Linux:**

```
echo $PATH | tr ':' '\n’
```

You'll see a list of directories — these are searched in order

# A Key Debugging Skill

**"Command not found" almost always means:**

The program exists, but its directory isn't in your PATH

**The fix:**

1. Find where the program was installed
2. Add that directory to PATH
3. Or use the full path to run it

# Check for Python

```
python --version
```

If that doesn't work:

```
python3 --version
```

Find where it lives:

Windows: `where python`

Mac: `which python3`

# Warning: Microsoft Store Python Can Cause Errors

If you run `where python` and see:

```
C:\Users\Student\AppData\Local\Microsoft\WindowsApps\python.exe
```

If where python returns a path containing WindowsApps, this is a Microsoft Store shortcut, not a working Python installation.

You'll need to install Python from python.org and ensure 'Add Python to PATH' is checked during installation.

# Part 2:
# What is Git?
# What is GitHub?

# Git vs GitHub:

| Git | GitHub |
| --- | --- |
| THE VERSION CONTROL SYSTEM | THE HOSTING PLATFORM |
| A free, open-source tool that runs on your computer to track changes in files. | A cloud service that hosts Git repositories online for collaboration and sharing. |
| · Created by Linus Torvalds (2005) | · Owned by Microsoft (since 2018) |
| · Works locally, no internet needed | · Web interface + collaboration tools |
| · Command-line interface | · Alternatives: GitLab, Bitbucket |

# Core Vocabulary

| Term | Definition |
| --- | --- |
| **Repository (repo)** | A project folder tracked by Git. Contains all your files plus the complete history of every change ever made. |
| **Commit** | A snapshot of your changes at a specific point in time. Each commit has a message describing what changed and why. |
| **Branch** | A parallel version of your repository. Work on features or experiments without affecting the main codebase. |
| **Push / Pull** | **Push:** Send your local commits to GitHub.<br>**Pull:** Download changes from GitHub to your computer. |
| **Clone** | Download a complete copy of a repository from GitHub to your computer, including its full history. |
| **Fork** | Create your own copy of someone else's repository on GitHub. You can modify it without affecting the original. |

# Check for Git

```
git --version
```

**See a version number?**

Skip to configuration

**See "command not found"?**

Check Path

or

Need to install

# Installing Git

**Windows:**

1. Download from git-scm.com/download/windows
2. Run installer, accept defaults
3. Important: Choose "Git from command line and 3rd-party software"
4. Close and reopen PowerShell

**Mac:**

Option A: macOS prompts to install when you run `git --version`

Option B: `brew install git`

# Configure Git

Tell Git who you are:

```
git config --global user.name "Your User Name“
git config --global user.email you@georgetown.edu
git config --global init.defaultBranch main
```

**Use your real email:** the same one you'll use for GitHub

Verify:

```
git config --list
```

# Configure Git: Set an Editor

Tell Git which editor you want to use
(It might default to vim):

```
git config --global core.editor "notepad" # Windows
git config --global core.editor "nano" # Mac
```

# Configure Git: Set an Editor

Tell Git who you are:

```
git config --global core.editor "notepad" # Windows
git config --global core.editor "nano" # Mac
```

**Use your real email:** the same one you'll use for GitHub

Verify:

```
git config --list
```

# Part 3:
# Connect Your GitHub Account

# GitHub Account

If you don’t have a GitHub Account:

1. Go to github.com - Click "Sign Up" in the top right corner
2. Enter your information - Email, username, and password
3. Verify your email - Check inbox for confirmation link
4. Select free plan - The free tier has everything we need

**Student Developer Pack:** Apply at education.github.com for free Pro features including GitHub Copilot

# Why SSH Keys?

GitHub no longer accepts passwords for Git operations

**SSH Keys:**
More secure than passwords

- No need to enter credentials repeatedly
- Industry standard practice
- Your computer proves its identity with a cryptographic key pair

Keys are regularly used in cloud services (we will use them with AWS)

# Generate SSH Key

Step 1: Check for existing keys

```
ls -la ~/.ssh
```

Step 2: Generate new key (if needed)

```
ssh-keygen -t ed25519 -C you@georgetown.edu
```

Press Enter for default location

# Add Key to SSH Agent

Mac/Linux:

```
eval "$(ssh-agent -s)“
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Windows:

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

# Add Key to GitHub

**Copy your public key:**

Mac/Linux: `pbcopy < ~/.ssh/id_ed25519.pub`

Windows: `Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub -RAW | Set-Clipboard`

**On GitHub:**

1. Profile → Settings → SSH and GPG keys
2. New SSH key
3. Paste and save

# Test SSH Connection

```
ssh -T git@github.com
```

First time: Type "yes" when asked about authenticity
Success looks like:

"Hi username! You've successfully authenticated..."

# Part 4:
# Create Your Repository and GitHub Page

# Create Your Repository

**Steps:**

- **Click the "+" icon → "New repository"** - Or go to github.com/new
- **Repository name: username.github.io** - Replace "username" with your actual GitHub username
- **Set visibility to "Public"** - GitHub Pages requires public repos (free tier)
- **Check "Add a README file"** - This initializes the repo with content
- **Click "Create repository“**

**WHY THIS NAME?** The special name pattern username.github.io tells GitHub to automatically publish this repository as a website.

Your site will be live at: **https://username.github.io**

# What is GitHub Pages?

| WHAT IT IS | COMMON USES | LIMITATIONS |
| --- | --- | --- |
| · Turns your repository into a website | · Personal portfolios | · Static only (no databases) |
| · Automatic HTTPS (secure) | · Project documentation | · No server-side code |
| · Custom domain support | · Course/research websites | · 1GB storage limit |
| · Updates when you push changes | · Data visualizations & dashboards | · Public repos only (free tier) |

**"Static"** means HTML, CSS, JavaScript: everything runs in the browser. No Python, R, or database queries on the server.

# Enabling GitHub Pages

**Steps:**

- **Go to repository Settings** - Click the "Settings" tab in your repo
- **Navigate to "Pages" in sidebar** - Under "Code and automation" section
- **Under "Source", select branch** - Choose main and / (root)
- **Click "Save"** - Your site will deploy in 1-2 minutes

**YOUR SITE URL:** `https://username.github.io`
Because you named your repo username.github.io, your site lives at the root URL. Other repos would be at /repo-name

**DEPLOYMENT TIME:** First deploy: ~1-2 minutes. Subsequent updates: usually faster.

# Creating an index.html

Sample site on the Canvas page

# Uploading & Committing Your File

Uploading & Committing Your File

**Steps:**

1. In your repository, click "Add file“, Then select "Upload files"
2. Drag your index.html file - Or click "choose your files" to browse
3. Write a commit message - Describe what you're adding
4. Click "Commit changes" - Your site will update in ~1 minute

# README.md vs. index.html

You now have two files that could serve as a “homepage”, but they serve different audiences.

| File | Where it appears | When it’s displayed | Audience | Primary use |
|---|---|---|---|---|
| README.md | GitHub | When viewing the repository on github.com | Developers and collaborators | Project description, setup instructions, contributor guidelines, documentation |
| index.html | Your website | When visiting your GitHub Pages site | General public | Portfolio, project showcase, public-facing content |

GitHub repository URL: `github.com/username/username.github.io`
GitHub Pages site URL: `username.github.io`

Keep both. They serve complementary purposes and do not conflict.